Resources

rand

Uniformly distributed random numbers

Syntax

```
X = rand
X = rand(n)
X = rand(sz1,...,szN)
X = rand(sz)

X = rand( __ ,typename)
X = rand( __ ,like=p)

X = rand(s, __ )
```

Description

`X = rand` returns a random scalar drawn from the uniform distribution in the interval (0,1).

`X = rand(n)` returns an n-by-n matrix of uniformly distributed random numbers. example

`X = rand(sz1,...,szN)` returns an `sz1`-by-...-by-`szN` array of random numbers where `sz1,...,szN` indicate the size of each dimension. For example, `rand(3,4)` returns a 3-by-4 matrix. example

`X = rand(sz)` returns an array of random numbers where size vector `sz` defines `size(X)`. For example, `rand([3 4])` returns a 3-by-4 matrix. example

`X = rand( __ ,typename)` returns an array of random numbers of data type `typename`. The `typename` input can be either `"single"` or `"double"`. You can use any of the input arguments in the previous syntaxes. example

`X = rand( __ ,like=p)` returns an array of random numbers like `p`; that is, of the same data type and complexity (real or complex) as `p`. You can specify either `typename` or `like`, but not both. example

`X = rand(s, __ )` generates numbers from random number stream `s` instead of the default global stream. To create a stream, use `RandStream`. You can specify `s` followed by any of the input argument combinations in previous syntaxes.

Examples

collapse all

Matrix of Random Numbers

Generate a 5-by-5 matrix of uniformly distributed random numbers between 0 and 1.

Open in MATLAB Online

Copy Command

```
r = rand(5)
```

Get

```
r = 5×5

    0.8147    0.0975    0.1576    0.1419    0.6557
    0.9058    0.2785    0.9706    0.4218    0.0357
    0.1270    0.5469    0.9572    0.9157    0.8491
    0.9134    0.9575    0.4854    0.7922    0.9340
    0.6324    0.9649    0.8003    0.9595    0.6787
```

Random Numbers Within Specified Interval

Generate a 10-by-1 column vector of uniformly distributed numbers in the interval (-5,5). You can generate `n` random numbers in the interval (a,b) with the formula `r = a + (b-a).*rand(n,1)`.

Open in MATLAB Online

Copy Command

```
a = -5;
b = 5;
n = 10;
r = a + (b-a).*rand(n,1)
```

Get

```
r = 10×1

    3.1472
    4.0579
   -3.7301
    4.1338
    1.3236
   -4.0246
   -2.2150
    0.4688
    4.5751
    4.6489
```

Random Integers

Use the `randi` function (instead of `rand`) to generate 5 random integers from the uniform distribution between 10 and 50.

Open in MATLAB Online

Copy Command

```
r = randi([10 50],1,5)
```

Get

```
r = 1×5
```

```
    43    47    15    47    35
```

Reset Random Number Generator

Save the current state of the random number generator and create a 1-by-5 vector of random numbers.

Open in MATLAB Online

Copy Command

```
s = rng;
r = rand(1,5)
```

Get

```
r = 1×5

    0.8147    0.9058    0.1270    0.9134    0.6324
```

Restore the state of the random number generator to `s`, and then create a new 1-by-5 vector of random numbers. The values are the same as before.

```
rng(s);
r1 = rand(1,5)
```

Get

```
r1 = 1×5

    0.8147    0.9058    0.1270    0.9134    0.6324
```

3-D Array of Random Numbers

Create a 3-by-2-by-3 array of random numbers.

Open in MATLAB Online

Copy Command

```
X = rand([3,2,3])
```

Get

```
X =
X(:,:,1) =

    0.8147    0.9134
    0.9058    0.6324
    0.1270    0.0975


X(:,:,2) =
```

```
    0.2785    0.9649
    0.5469    0.1576
    0.9575    0.9706


X(:,:,3) =

    0.9572    0.1419
    0.4854    0.4218
    0.8003    0.9157
```

Specify Data Type of Random Numbers

Create a 1-by-4 vector of random numbers whose elements are single precision.

Open in MATLAB Online

Copy Command

```
r = rand(1,4,"single")
```

Get

```
r = 1×4 single row vector

    0.8147    0.9058    0.1270    0.9134
```

```
class(r)
```

Get

```
ans =
'single'
```

Size Defined by Existing Array

Create a matrix of uniformly distributed random numbers with the same size as an existing array.

Open in MATLAB Online

Copy Command

```
A = [3 2; -2 1];
sz = size(A);
X = rand(sz)
```

Get

```
X = 2×2

    0.8147    0.1270
    0.9058    0.9134
```

It is a common pattern to combine the previous two lines of code into a single line:

```
X = rand(size(A));
```

Get

Size and Data Type Defined by Existing Array

Create a 2-by-2 matrix of single-precision random numbers.

Open in MATLAB Online

Copy Command

```
p = single([3 2; -2 1]);
```

Get

Create an array of random numbers that is the same size and data type as p.

```
X = rand(size(p),like=p)
```

Get

```
X = 2×2 single matrix

    0.8147    0.1270
    0.9058    0.9134
```

```
class(X)
```

Get

```
ans =
'single'
```

Random Complex Numbers

Generate 10 random complex numbers from the uniform distribution over a square domain with real and imaginary parts in the interval (0,1).

Open in MATLAB Online

Copy Command

```
a = rand(10,1,like=1i)
```

Get

```
a = 10×1 complex

   0.8147 + 0.9058i
   0.1270 + 0.9134i
   0.6324 + 0.0975i
   0.2785 + 0.5469i
   0.9575 + 0.9649i
   0.1576 + 0.9706i
   0.9572 + 0.4854i
   0.8003 + 0.1419i
   0.4218 + 0.9157i
```

```
0.7922 + 0.9595i
```

Input Arguments

collapse all

`n` — Size of square matrix
integer value

Size of square matrix, specified as an integer value.

- If `n` is `0`, then `X` is an empty matrix.
- If `n` is negative, then it is treated as `0`.

Data Types: `single` | `double` | `int8` | `int16` | `int32` | `int64` | `uint8` | `uint16` | `uint32` | `uint64`

`sz1,...,szN` — Size of each dimension (as separate arguments)
integer values

Size of each dimension, specified as separate arguments of integer values.

- If the size of any dimension is `0`, then `X` is an empty array.
- If the size of any dimension is negative, then it is treated as `0`.
- Beyond the second dimension, `rand` ignores trailing dimensions with a size of 1. For example, `rand(3,1,1,1)` produces a 3-by-1 vector of random numbers.

Data Types: `single` | `double` | `int8` | `int16` | `int32` | `int64` | `uint8` | `uint16` | `uint32` | `uint64`

`sz` — Size of each dimension (as a row vector)
integer values

Size of each dimension, specified as a row vector of integer values. Each element of this vector indicates the size of the corresponding dimension:

- If the size of any dimension is `0`, then `X` is an empty array.
- If the size of any dimension is negative, then it is treated as `0`.
- Beyond the second dimension, `rand` ignores trailing dimensions with a size of 1. For example, `rand([3 1 1 1])` produces a 3-by-1 vector of random numbers.

Example: `sz = [2 3 4]` creates a 2-by-3-by-4 array.

Data Types: `single` | `double` | `int8` | `int16` | `int32` | `int64` | `uint8` | `uint16` | `uint32` | `uint64`

`typename` — Data type (class) to create
`"double"` (default) | `"single"`

Data type (class) to create, specified as `"double"`, `"single"`, or the name of another class that provides `rand` support.

Example: `rand(5,"single")`

p — Prototype of array to create
numeric array

Prototype of array to create, specified as a numeric array.

Example: `rand(5,like=p)`

Data Types: `single` | `double`
Complex Number Support: Yes

s — Random number stream
`RandStream` object

Random number stream, specified as a `RandStream` object.

Example: `s = RandStream("dsfmt19937"); rand(s,[3 1])`

Output Arguments

collapse all

X — Output array
scalar | vector | matrix | multidimensional array

Output array, returned as a scalar, vector, matrix, or multidimensional array.

More About

collapse all

Pseudorandom Number Generator

The underlying number generator for `rand` is a pseudorandom number generator, which creates a deterministic sequence of numbers that appear random. These numbers are predictable if the seed and the deterministic algorithm of the generator are known. While not truly random, the generated numbers pass various statistical tests of randomness, satisfying the independent and identically distributed (i.i.d.) condition, and justifying the name *pseudorandom*.

Tips

- The sequence of numbers produced by `rand` is determined by the internal settings of the uniform pseudorandom number generator that underlies `rand`, `randi`, and `randn`. You can control that shared random number generator using `rng`.

Extended Capabilities

C/C++ Code Generation
Generate C and C++ code using MATLAB® Coder™.

GPU Code Generation
Generate CUDA® code for NVIDIA® GPUs using GPU Coder™.

Thread-Based Environment
Run code in the background using MATLAB® `backgroundPool` or accelerate code with Parallel Computing Toolbox™ `ThreadPool`.

GPU Arrays
Accelerate code by running on a graphics processing unit (GPU) using Parallel Computing Toolbox™.

Distributed Arrays
Partition large arrays across the combined memory of your cluster using Parallel Computing Toolbox™.

Version History

expand all

Introduced before R2006a

R2022a: Match complexity with `like`, and use `like` with `RandStream` object

R2014a: Match data type of an existing variable with `'like'`

R2013b: Non-integer size inputs are not supported⚠

R2008b: `'seed'`, `'state'`, and `'twister'` inputs are not recommended⚠

See Also

`randi` | `randn` | `rng` | `RandStream` | `sprand` | `sprandn` | `randperm`

Topics

Create Arrays of Random Numbers
Controlling Random Number Generation
Random Numbers Within a Specific Range
Class Support for Array-Creation Functions
Why Do Random Numbers Repeat After Startup?

YOU MIGHT ALSO BE INTERESTED IN

Converting Deep Learning Models Between PyTorch, TensorFlow, and MATLAB

Import and export AI models between PyTorch®, TensorFlow™, and MATLAB®.

Learn more